Fleet Management · Dados para o passo a passo do Motorista

Tudo pronto para copiar e colar: caminho triste primeiro, depois o caminho feliz. Cada bloco traz o comando de banco para mostrar a persistência.

Dados de partida (já existem no banco)

Item	Valor
Motorista (você)	Pablo Driver · pablo.driver@velocityfleet.com · senha 12345678
x-api-key do motorista	`956e1dbe05ec48fe81b3f8c98a07640c612ae21e089c821d267a61b9ef4eb6be`
driverId (Pablo)	`0548bc21-2a86-420a-9733-b4cec146e975`
Veículo livre (Fiat Strada, FLT1A23, 82000 km)	`e0cfd550-2055-4484-90bd-6e03c1b7d559`
Outro motorista	Carlos Driver · driverId `11c97ad5-1239-4430-b1c0-9275f3bff352`
Veículo em uso (VW Saveiro, FLT2B34)	`c0982d4f-becc-4c6f-ab6a-c2f38f9eef10`
Viagem do outro motorista (IN_PROGRESS)	`85cf6289-716c-4340-9e84-df3358eac806`

Base local: `http://localhost:3000` (Swagger em `/api/docs`). Nos comandos de banco, o contêiner é `fleet_postgres`. O caminho feliz muda o hodômetro do veículo livre (+120 km a cada execução completa). O dado do outro motorista é fixo para os testes do caminho triste. Este documento contém credenciais de demonstração: não o publique.

Parte 1 · Caminho triste

T1 · Login sem a API key

POST `/auth/login`

HEADERS

```
Content-Type: application/json
(não envie o header x-api-key)
```

BODY (COPIAR E COLAR)

```
{
  "email": "pablo.driver@velocityfleet.com",
  "password": "12345678"
}
```

Resultado esperado: 401 Unauthorized

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT email, api_key_last_used_at FROM users
WHERE email='pablo.driver@velocityfleet.com';"
```

Nada mudou: o último uso da API key continua vazio.

T2 · Login com senha errada

POST `/auth/login`

HEADERS

```
Content-Type: application/json
x-api-key: 956e1dbe05ec48fe81b3f8c98a07640c612ae21e089c821d267a61b9ef4eb6be
```

BODY (COPIAR E COLAR)

```
{
  "email": "pablo.driver@velocityfleet.com",
  "password": "senha-errada-1"
}
```

Resultado esperado: 401 Unauthorized

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT email, api_key_last_used_at FROM users
WHERE email='pablo.driver@velocityfleet.com';"
```

Continua sem registro de uso: o login falhou.

T3 · Login com API key inválida

POST `/auth/login`

HEADERS

```
Content-Type: application/json
x-api-key: chave-invalida
```

BODY (COPIAR E COLAR)

```
{
  "email": "pablo.driver@velocityfleet.com",
  "password": "12345678"
}
```

Resultado esperado: 401 Unauthorized

Preparação · Login válido (para obter o token das próximas etapas)

POST `/auth/login`

HEADERS

```
Content-Type: application/json
x-api-key: 956e1dbe05ec48fe81b3f8c98a07640c612ae21e089c821d267a61b9ef4eb6be
```

BODY (COPIAR E COLAR)

```
{
  "email": "pablo.driver@velocityfleet.com",
  "password": "12345678"
}
```

Resultado esperado: 200 OK. Copie o accessToken e use em todas as próximas chamadas: Authorization: Bearer <accessToken>

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT email, api_key_last_used_at FROM users
WHERE email='pablo.driver@velocityfleet.com';"
```

Agora o último uso da API key foi gravado (persistência do login).

T4 · Motorista tenta listar usuários

GET `/users`

HEADERS

```
Authorization: Bearer <accessToken>
```

Resultado esperado: 403 Forbidden

T5 · Motorista tenta cadastrar veículo

POST `/vehicles`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "plate": "XYZ1A23",
  "model": "Fiat Strada",
  "year": 2023,
  "currentMileage": 1000
}
```

Resultado esperado: 403 Forbidden

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT count(*) FROM vehicles WHERE plate='XY
Z1A23';"
```

Resultado 0: o veículo não foi gravado.

T6 · Criar viagem com o driverId de outro motorista

POST `/trips`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "driverId": "11c97ad5-1239-4430-b1c0-9275f3bff352",
  "vehicleId": "e0cfd550-2055-4484-90bd-6e03c1b7d559",
  "startLocation": "01310-100",
  "endLocation": "20040-020"
}
```

Resultado esperado: 403 Forbidden

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT count(*) FROM trips WHERE fk_driver_id
='11c97ad5-1239-4430-b1c0-9275f3bff352' AND fk_vehicle_id='e0cfd550-2055-4484-90bd-6e03c1b7d559';"
```

Resultado 0: nenhuma viagem foi criada para o outro motorista.

T7 · Criar viagem com CEP inexistente

POST `/trips`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "driverId": "0548bc21-2a86-420a-9733-b4cec146e975",
  "vehicleId": "e0cfd550-2055-4484-90bd-6e03c1b7d559",
  "startLocation": "00000-000",
  "endLocation": "20040-020"
}
```

Resultado esperado: 400 Bad Request (CEP inválido no ViaCEP)

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT count(*) FROM trips WHERE fk_driver_id
='0548bc21-2a86-420a-9733-b4cec146e975';"
```

Resultado 0: a viagem não foi gravada.

T8 · Criar viagem com veículo já em uso

POST `/trips`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "driverId": "0548bc21-2a86-420a-9733-b4cec146e975",
  "vehicleId": "c0982d4f-becc-4c6f-ab6a-c2f38f9eef10",
  "startLocation": "01310-100",
  "endLocation": "20040-020"
}
```

Resultado esperado: 409 Conflict (veículo em uso pela viagem do outro motorista)

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT plate, status FROM vehicles WHERE id_v
ehicle='c0982d4f-becc-4c6f-ab6a-c2f38f9eef10';"
```

O veículo segue IN_USE, com a viagem do outro motorista.

T9 · Criar viagem enviando startKm

POST `/trips`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "driverId": "0548bc21-2a86-420a-9733-b4cec146e975",
  "vehicleId": "e0cfd550-2055-4484-90bd-6e03c1b7d559",
  "startKm": 90000,
  "startLocation": "01310-100",
  "endLocation": "20040-020"
}
```

Resultado esperado: 400 Bad Request (startKm não é aceito)

T10 · Ler a viagem de outro motorista

GET `/trips/85cf6289-716c-4340-9e84-df3358eac806`

HEADERS

```
Authorization: Bearer <accessToken>
```

Resultado esperado: 404 Not Found (para o motorista, o dado alheio não existe)

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT id_trip, status, fk_driver_id FROM tri
ps WHERE id_trip='85cf6289-716c-4340-9e84-df3358eac806';"
```

A viagem existe no banco, mas pertence a Carlos Driver.

T11 · Iniciar a viagem de outro motorista

PATCH `/trips/85cf6289-716c-4340-9e84-df3358eac806/start`

HEADERS

```
Authorization: Bearer <accessToken>
```

Resultado esperado: 404 Not Found

T12 · Iniciar viagem enviando body (currentMileage)

PATCH `/trips/85cf6289-716c-4340-9e84-df3358eac806/start`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "currentMileage": 90000
}
```

Resultado esperado: 400 Bad Request (o start não recebe body)

T13 · Finalizar com endKm zero

PATCH `/trips/85cf6289-716c-4340-9e84-df3358eac806/end`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "endKm": 0,
  "endLocation": "20040-020"
}
```

Resultado esperado: 400 Bad Request (endKm entre 1 e 100000)

T14 · Finalizar com endKm acima do limite

PATCH `/trips/85cf6289-716c-4340-9e84-df3358eac806/end`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "endKm": 100001,
  "endLocation": "20040-020"
}
```

Resultado esperado: 400 Bad Request

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT status, end_km FROM trips WHERE id_tri
p='85cf6289-716c-4340-9e84-df3358eac806';"
```

A viagem alheia segue IN_PROGRESS, sem end_km.

T15 · Abastecer enviando mileage

POST `/refuelings`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "vehicleId": "e0cfd550-2055-4484-90bd-6e03c1b7d559",
  "driverId": "0548bc21-2a86-420a-9733-b4cec146e975",
  "mileage": 90000,
  "litersAdded": 10,
  "costPerLiter": 5.5,
  "fuelType": "GASOLINE"
}
```

Resultado esperado: 400 Bad Request (o campo mileage não existe mais)

T16 · Abastecer veículo que está em viagem de outro motorista

POST `/refuelings`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "vehicleId": "c0982d4f-becc-4c6f-ab6a-c2f38f9eef10",
  "driverId": "0548bc21-2a86-420a-9733-b4cec146e975",
  "litersAdded": 10,
  "costPerLiter": 5.5,
  "fuelType": "GASOLINE"
}
```

Resultado esperado: 409 Conflict (o motorista não é o da viagem ativa do veículo)

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT count(*) FROM refuelings WHERE fk_vehi
cle_id='c0982d4f-becc-4c6f-ab6a-c2f38f9eef10';"
```

Resultado 0: nada foi gravado.

T17 · Incidente com tipo inválido

POST `/incidents`

HEADERS

```
Authorization: Bearer <accessToken>
Content-Type: multipart/form-data (no Swagger, preencha os campos abaixo)
```

CAMPOS DO FORMULÁRIO (COPIAR E COLAR)

```
vehicleId: e0cfd550-2055-4484-90bd-6e03c1b7d559
driverId: 0548bc21-2a86-420a-9733-b4cec146e975
type: BATIDA
severity: LOW
description: Teste de tipo inválido
```

Resultado esperado: 400 Bad Request (tipos válidos: ACCIDENT, MECHANICAL_FAILURE, OTHER)

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT count(*) FROM incidents WHERE fk_drive
r_id='0548bc21-2a86-420a-9733-b4cec146e975';"
```

Resultado 0: o incidente não foi gravado.

T18 · Apagar veículo de vez

DELETE `/vehicles/e0cfd550-2055-4484-90bd-6e03c1b7d559/permanent`

HEADERS

```
Authorization: Bearer <accessToken>
```

Resultado esperado: 403 Forbidden (deleção total é só do ADMIN)

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT plate, status, current_mileage FROM ve
hicles WHERE id_vehicle='e0cfd550-2055-4484-90bd-6e03c1b7d559';"
```

O veículo continua no banco.

T19 · Motorista tenta subir o próprio papel

PATCH `/users/928bb76e-dfa0-4694-8c64-db7fd6ff01a8/role`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "roleId": "00000000-0000-0000-0000-000000000000"
}
```

Resultado esperado: 403 Forbidden

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT u.email, r.name AS papel FROM users u
JOIN roles r ON r.id_role=u.fk_role_id WHERE u.email='pablo.driver@velocityfleet.com';"
```

O papel continua DRIVER.

Parte 2 · Caminho feliz

Use o mesmo accessToken do login. Onde aparecer <TRIP_ID>, cole o id devolvido no passo F3.

F1 · Login

POST `/auth/login`

HEADERS

```
Content-Type: application/json
x-api-key: 956e1dbe05ec48fe81b3f8c98a07640c612ae21e089c821d267a61b9ef4eb6be
```

BODY (COPIAR E COLAR)

```
{
  "email": "pablo.driver@velocityfleet.com",
  "password": "12345678"
}
```

Resultado esperado: 200 OK com accessToken, user.permissions e user.driverId

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT email, is_active, api_key_last_used_at
FROM users WHERE email='pablo.driver@velocityfleet.com';"
```

O usuário existe, está ativo e o último uso da API key foi gravado.

F2 · Ver o próprio perfil

GET `/users/me`

HEADERS

```
Authorization: Bearer <accessToken>
```

Resultado esperado: 200 OK com driverId 0548bc21-2a86-420a-9733-b4cec146e975

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT u.full_name, r.name AS papel, d.id_dri
ver, d.license_expiry FROM users u JOIN roles r ON r.id_role=u.fk_role_id JOIN drivers d ON d.fk_user_id=
u.id_user WHERE u.email='pablo.driver@velocityfleet.com';"
```

Usuário, papel DRIVER e o perfil de motorista vinculado.

F3 · Planejar a viagem

POST `/trips`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "driverId": "0548bc21-2a86-420a-9733-b4cec146e975",
  "vehicleId": "e0cfd550-2055-4484-90bd-6e03c1b7d559",
  "startLocation": "01310-100",
  "endLocation": "20040-020"
}
```

Resultado esperado: 201 Created, status PLANNED. Copie o id devolvido: é o <TRIP_ID> dos próximos passos

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT id_trip, status, start_km, end_km, sta
rt_location, end_location FROM trips WHERE fk_driver_id='0548bc21-2a86-420a-9733-b4cec146e975' ORDER BY c
reated_at DESC LIMIT 1;"
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT plate, status, current_mileage FROM ve
hicles WHERE id_vehicle='e0cfd550-2055-4484-90bd-6e03c1b7d559';"
```

Viagem PLANNED com start_km igual ao hodômetro (82000) e veículo IN_USE.

F4 · Iniciar a viagem

PATCH `/trips/<TRIP_ID>/start`

HEADERS

```
Authorization: Bearer <accessToken>
```

Resultado esperado: 200 OK, status IN_PROGRESS (sem body)

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT id_trip, status, start_km, end_km, sta
rt_location, end_location FROM trips WHERE fk_driver_id='0548bc21-2a86-420a-9733-b4cec146e975' ORDER BY c
reated_at DESC LIMIT 1;"
```

Status IN_PROGRESS e start_time preenchido.

F5 · Abastecer durante a viagem

POST `/refuelings`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "vehicleId": "e0cfd550-2055-4484-90bd-6e03c1b7d559",
  "driverId": "0548bc21-2a86-420a-9733-b4cec146e975",
  "litersAdded": 10.5,
  "costPerLiter": 5.89,
  "fuelType": "GASOLINE"
}
```

Resultado esperado: 201 Created, totalCost 61.85

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT liters_added, cost_per_liter, total_co
st, fuel_type, mileage FROM refuelings WHERE fk_driver_id='0548bc21-2a86-420a-9733-b4cec146e975' ORDER BY
created_at DESC LIMIT 1;"
```

Custo total calculado pelo servidor; o mileage é o hodômetro do veículo no momento.

F6 · Registrar um incidente

POST `/incidents`

HEADERS

```
Authorization: Bearer <accessToken>
Content-Type: multipart/form-data (no Swagger, preencha os campos abaixo)
```

CAMPOS DO FORMULÁRIO (COPIAR E COLAR)

```
vehicleId: e0cfd550-2055-4484-90bd-6e03c1b7d559
driverId: 0548bc21-2a86-420a-9733-b4cec146e975
tripId: <TRIP_ID>
type: MECHANICAL_FAILURE
severity: LOW
description: Luz de aviso do painel acendeu durante a viagem
```

Resultado esperado: 201 Created (a foto é opcional)

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT type, severity, status, description FR
OM incidents WHERE fk_driver_id='0548bc21-2a86-420a-9733-b4cec146e975' ORDER BY created_at DESC LIMIT 1;"
```

Incidente gravado e ligado à viagem.

F7 · Finalizar a viagem (km rodados)

PATCH `/trips/<TRIP_ID>/end`

HEADERS

```
Authorization: Bearer <accessToken>
```

BODY (COPIAR E COLAR)

```
{
  "endKm": 120,
  "endLocation": "20040-020"
}
```

Resultado esperado: 200 OK, status COMPLETED, distanceKm 120

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT id_trip, status, start_km, end_km, sta
rt_location, end_location FROM trips WHERE fk_driver_id='0548bc21-2a86-420a-9733-b4cec146e975' ORDER BY c
reated_at DESC LIMIT 1;"
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT plate, status, current_mileage FROM ve
hicles WHERE id_vehicle='e0cfd550-2055-4484-90bd-6e03c1b7d559';"
```

Viagem COMPLETED com end_km = 82120; veículo AVAILABLE com hodômetro 82120 (82000 + 120).

F8 · Consultar o resultado

GET `/trips/<TRIP_ID>`

HEADERS

```
Authorization: Bearer <accessToken>
```

Resultado esperado: 200 OK. Também: GET /trips lista só as viagens do próprio motorista

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT count(*) AS minhas_viagens FROM trips
WHERE fk_driver_id='0548bc21-2a86-420a-9733-b4cec146e975';"
```

Quantidade de viagens do Pablo Driver.

F9 · Alternativa: planejar outra viagem e cancelar

PATCH `/trips/<TRIP_ID>/cancel`

HEADERS

```
Authorization: Bearer <accessToken>
```

Resultado esperado: 200 OK, status CANCELLED. Antes, crie outra viagem com o body do passo F3 e use o id novo

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT id_trip, status FROM trips WHERE fk_dr
iver_id='0548bc21-2a86-420a-9733-b4cec146e975' ORDER BY created_at DESC LIMIT 2;"
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT plate, status, current_mileage FROM ve
hicles WHERE id_vehicle='e0cfd550-2055-4484-90bd-6e03c1b7d559';"
```

A última viagem está CANCELLED e o veículo voltou a AVAILABLE.

F10 · Prova de auditoria

Resultado esperado: Toda mudança de viagem, abastecimento e incidente gera linha imutável em audit_logs

COMANDO NO BANCO (PROVA DE PERSISTÊNCIA)

```
docker exec fleet_postgres psql -U postgres -d fleet_db -c "SELECT entity_type, action, created_at FROM a
udit_logs WHERE fk_user_id='928bb76e-dfa0-4694-8c64-db7fd6ff01a8' ORDER BY created_at;"
```

Uma linha para cada ação do motorista (criar, iniciar, abastecer, incidente, finalizar, cancelar).